

Active Caching for Similarity Queries Based on Shared-Neighbor Information

Michael E. Houle
National Institute of
Informatics
2-1-2 Hitotsubashi,
Chiyoda-ku
Tokyo 101-8430, Japan
meh@nii.ac.jp

Vincent Oria
College of Computing
Sciences
New Jersey Institute of
Technology
University Heights
Newark, NJ 07102, USA
oria@njit.edu

Umar Qasim
College of Computing
Sciences
New Jersey Institute of
Technology
University Heights
Newark, NJ 07102, USA
muq2@njit.edu

ABSTRACT

Novel applications such as recommender systems, uncertain databases, and multimedia databases are designed to process similarity queries that produce ranked lists of objects as their results. Similarity queries typically result in disk access latency and incur a substantial computational cost. In this paper, we propose an ‘active caching’ technique for similarity queries that is capable of synthesizing query results from cached information even when the required result list is not explicitly stored in the cache. Our solution, the Cache Estimated Significance (CES) model, is based on shared-neighbor similarity measures, which assess the strength of the relationship between two objects as a function of the number of other objects in the common intersection of their neighborhoods. The proposed method is general in that it does not require that the features be drawn from a metric space, nor does it require that the partial orders induced by the similarity measure be monotonic. Experimental results on real data sets show a substantial cache hit rate when compared with traditional caching approaches.

Categories and Subject Descriptors

H.3.3 [Information Storage and Retrieval]: Information Search and Retrieval—*retrieval models, search process*

General Terms

Performance

1. INTRODUCTION

Caching made its first appearance in 1967 as a memory improvement in Model 85 of the IBM System/360 product line [20], and has seen many important applications since. A CPU cache helps expedite the access of data that would otherwise need to be fetched from main memory. A page cache,

usually managed by the operating system kernel, helps to reduce the amount of fetching from disk. Database caching can substantially improve the throughput of database applications. The caching of web content reduces network traffic, server load, and latency. More recently, semantic caching techniques have been developed to store the results of prior queries in order to reuse them fully or partially to answer related queries [4]. Semantic caching can improve the performance substantially when a series of semantically-associated queries are posed, provided that many of the queries have results that intersect (or are contained within) cached results.

Caching strategies can be divided into two broad classes: traditional ‘passive’ caching, and the more recent ‘active’ caching. With passive caching, where the server query result is retrieved either directly from the cache or from the disk, the effectiveness of the operation is guaranteed. Passive cache management strategies generally seek to fill the cache with result lists for the most popular queries, and to utilize effective replacement strategies to maximize the overall performance. One example of a retrieval system for which passive caching may be effective is the Excite search engine — Markatos and Chronaki discovered a large number of frequently-posed queries in the retrieval logs that constitute excellent candidates for caching, with a hit rate of up to 25% [18]. In experiments involving Digital Equipment Corporation query traces, passive caching was able to reduce latency by 22%–26% [13].

In general, only limited performance gains are possible with passive caching. Active caching techniques attempt to improve upon the performance of passive caching by synthesizing a query result from stored information whenever the sought-after result is not explicitly present. This form of caching is referred to as ‘active’ since the cache can be considered to function in a limited query-processing role [17].

The active caching techniques proposed in the research literature to date are extensions of relational database caching strategies that aim to answer Boolean queries: the caching strategies attempt to make use of the stored results of past queries to generate a new result that satisfies the containment criteria of the current query. For similarity queries returning a ranked list of the top- k relevant objects — as often arise in applications involving search engines, recommender systems, information retrieval and multimedia databases — this form of active caching cannot be applied. One reason is

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

CIKM’10, October 26–30, 2010, Toronto, Ontario, Canada.
Copyright 2010 ACM 978-1-4503-0099-5/10/10 ...\$10.00.

that for top- k similarity queries, rather than attempting to list all possible objects satisfying the containment criteria, search engines return only a relatively small proportion of relevant objects [24].

In this paper, we propose a more general active caching strategy for top- k similarity queries (also known as k -nearest-neighbor, or k -NN queries). The method is based on ‘shared-neighbor’ similarity measures, which assess the similarity between two objects in terms of the number of other objects in the common intersection of their neighborhoods. The intersection size of neighborhood sets has been used as the basis of the merge criteria of several heuristics for data clustering [5, 6, 8, 11], as well as the relevant-set correlation (RSC) clustering model of [9], which uses shared-neighbor information to directly assess the quality of cluster candidates, and to rank the members of clusters in order of relevance. Our active caching solution uses techniques developed for RSC to derive ranking functions for active caching that take into account potential variation in the sizes of cached neighbor lists, without any knowledge of the similarity values used in producing these lists.

The remainder of the paper is organized as follows. In the next section, we give an overview of existing active caching methods. In Section 3, we briefly describe the original RSC model for clustering as well as supporting terminology and notation. In Section 4, we list the most commonly-used measures of shared-neighbor information, and describe how secondary similarity measures can be derived using techniques developed under the RSC model. In Section 5, we show how these similarity measures can be efficiently implemented. Experiments on real data sets are presented in Section 6 that show how the proposed active caching formulations can be surprisingly effective in terms of both recall and hit rate, even for relatively small cache sizes. Finally, the discussion is concluded in Section 7.

2. RELATED WORK

One well-known example of an active caching strategy is semantic caching, which assumes that the queries submitted to information retrieval systems are Boolean, and that the partial results of previous queries can be reused in composing results for new queries, using Boolean algebra. When the user submits an intersection query, it is decomposed into two parts: a probe query that fetches the relevant portion of the answer set from the cache, and a remainder query that fetches the missing part of the result from the server [3]. For a containment query, on the other hand, a remainder query need never be generated. Semantic caching is particularly attractive for use in mobile computing platforms, due to the resource limitations associated with mobile clients [14, 21, 26]. It has also been used in web caching to handle conjunctive queries supporting the use of Boolean operators AND and NOT between query terms [3]. However, the operator OR could not be supported, due to the exponential complexity of the semantic containment and intersection problem for full Boolean expressions. In [17], Luo, Naughton and Xue proposed an effective semantic caching scheme for Web search engines, and concluded that while answering cache-contained queries results in a significant performance gain, answering cache-intersecting queries is not worthwhile for top- k conjunctive term-based queries. The main limitation of semantic caching is that it can be applied only to Boolean queries, with the containment property being used to deter-

mine which queries can be answered from the cache. For these reasons, semantic caching techniques are not applicable for similarity queries.

Several studies have focused on search engine models where the cache acts in a query processing role. Saraiva et al. [23] proposed the storage of inverted lists of query terms (keywords) to assist in the active caching query-handling process. The cache mechanism uses a two-level scheme that combines cached query results and cached inverted lists. The results of repeated identical queries are cached at the front end, whereas data for frequently-used query terms are cached at a lower level. The inverted lists for each term are accessed, and used to generate lists of result documents containing all terms. In [16] a three-level cache was proposed wherein an intermediate level is added to the design. The intermediate level exploits frequently occurring pairs of terms by caching intersections or projections of the corresponding inverted lists. Although these combined caching strategies can increase the throughput of the system, they are only able to handle keyword-based queries. In general, k -NN queries cannot be accommodated, as all attributes are generally required for the computation of similarity values.

3. PRELIMINARIES

Before presenting the details of our proposed active caching strategy, we will introduce some of the required terminology, notation, and concepts.

3.1 Terminology

Let S be a data set drawn from some domain D . For every object $v \in S$, we assume the existence of a unique ordering $(v_1, v_2, \dots, v_{|S|})$ of the objects of S , where $i < j$ implies that v_i is more relevant (or similar) to v than v_j . With respect to v , the rank of object w ranges from 1 to $|S|$, and will be denoted by $rank(v, w)$. In practical settings, the object most relevant to v is generally v itself. Nevertheless, unless otherwise stated, we will not require that $rank(v, v) = 1$.

The relevance ranking for v induces a collection of distinct sets $Q(v, \lambda) = \{v_1, \dots, v_\lambda\}$ for each choice of set size $1 \leq \lambda \leq |S|$. With respect to the ranking, if a data set similarity query were to be based at object v , $Q(v, k)$ would represent the top- k relevant set. $Q(v, k)$ can also represent the result of a k -nearest neighbor (k -NN) query for v with respect to some unknown distance measure $dist : D \times D \rightarrow \mathbb{R}^{\geq 0}$. However, we will make no explicit use of the actual values of any distance function or other scoring function used to determine relevance rankings under the model.

Taken together over all $v \in S$, the rankings also induce a collection of inverted relevant sets $Q^{-1}(u, \lambda) = \{v \in S : u \in Q(v, \lambda)\}$ for every choice of $u \in S$ and $1 \leq \lambda \leq |S|$. It should be noted that whereas $j \neq \lambda$ implies that $Q(v, j)$ and $Q(v, \lambda)$ are distinct, it is possible that $Q^{-1}(u, j) = Q^{-1}(u, \lambda)$ for some choices of $u \in S$ and $j \neq \lambda$.

Consider now the situation in which a main-memory cache $\mathcal{C}(C, \lambda) \triangleq \{Q(v, \lambda) : v \in C\}$ of relevant sets of size λ is available for each object in a given subset $C \subseteq S$, for some fixed $\lambda \in [1, |S|]$. If each of the relevant sets is maintained as a list of objects sorted from most relevant to least relevant, the collection of relevant sets $\mathcal{C}(C, j)$ is also readily available for all $1 \leq j \leq \lambda$. We will accordingly refer to $\mathcal{C}(C, j)$ as a *subcache* of $\mathcal{C}(C, \lambda)$.

For a given $u \in S$, inverted relevant sets for u can also be

defined with respect to the cache $\mathcal{C}(C, \lambda)$, by restricting the membership of the lists to objects of C instead of S :

$$Q_C^{-1}(u, \lambda) \triangleq Q^{-1}(u, \lambda) \cap C = \{v \in C : u \in Q(v, \lambda)\}.$$

The collection of all such inverted lists taken over all choices of $u \in S$ will be referred to as the *inverted cache* corresponding to $\mathcal{C}(C, \lambda)$, and will be denoted by $\mathcal{C}^{-1}(C, \lambda) \triangleq \{Q_C^{-1}(v, \lambda) : v \in C\}$. We will use the terms *standard cache* and *standard relevant sets* (or more simply, *standard lists*) to refer to the original cache $\mathcal{C}(C, \lambda)$ and its lists, and the term *cache* loosely to refer to the set C taken together with its standard and inverted relevant sets.

3.2 Relevant Set Correlation

Set correlation can be regarded as a special case of the well-known Pearson correlation of variable pairs. Every object of some universal set Ω can be associated with a coordinate of a vector space whose dimension is equal to the size of S . A subset A of Ω can be represented by a zero-one characteristic vector in this space, where a coordinate value of 1 indicates that the corresponding object is a member of A , and a value of 0 indicates that the object does not belong to A . Even if no additional information is available regarding the nature of A and B , the relationship between A and B (and their underlying concepts) can be quantified in terms of the correlation between corresponding coordinates of their characteristic vectors.

For sequences of variables $(x_1, \dots, x_{|\Omega|})$ and $(y_1, \dots, y_{|\Omega|})$ with means $\bar{x}$ and $\bar{y}$, respectively, the standard Pearson sample correlation is given by the following formula [19]:

$$r = \frac{\sum_{i=1}^{|\Omega|} (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{|\Omega|} (x_i - \bar{x})^2 \sum_{i=1}^{|\Omega|} (y_i - \bar{y})^2}}.$$

Applying the formula to the characteristic vectors of sets $A, B \subseteq \Omega$, and noting that

$$\sum_{i=1}^{|\Omega|} x_i^2 = \sum_{i=1}^{|\Omega|} x_i = |\Omega| \cdot \bar{x}$$

whenever $x_i \in \{0, 1\}$, we obtain the following set correlation formula [9]:

$$R_\Omega(A, B) = \frac{|\Omega| \cdot \text{CM}(A, B) - \sqrt{|A| \cdot |B|}}{\sqrt{(|\Omega| - |A|)(|\Omega| - |B|)}},$$

where

$$\text{CM}(A, B) = \frac{|A \cap B|}{\sqrt{|A| \cdot |B|}}$$

is the popular *cosine* similarity measure between A and B [19]. Note that when the sizes of A and B are fixed, the set correlation value tends to the cosine measure value as the universal set size $|\Omega|$ increases.

Although the set correlation resembles to some extent the Spearman rank correlation and Kendall tau rank correlation coefficients appearing in the statistical literature [12], the latter two are fundamentally unsuited for application to relevant sets. Given two ranked lists of items drawn from a common domain, Spearman rank correlation assigns to each item an ordered pair of values equal to the rank of the item with respect to each ranked list. The Spearman rank correlation value is simply the Pearson correlation applied to

the collection of variable pairs. Compared to the set correlation, however, the Spearman rank correlation ascribes great importance to the magnitude of the difference between the ranks of a given item with respect to each list. The Kendall tau coefficient, on the other hand, is formulated in terms of the sign of the difference between the ranks of the item with respect to the two lists. For situations such as caching in which the available ranked lists span only a local neighborhood (the relevant set) and not the entire database, it is often the case that an individual item appears in one list and not the other, precluding the exact calculation of both Spearman and Kendall tau coefficients.

4. THE CES MODEL

With traditional caching strategies, in processing a top- k query object v for which results have not already been cached, the information is retrieved directly from disk. However, if the query result for v can be reliably estimated using only cached information and without performing expensive disk access operations, the computational savings may be considerable. In this section, we propose the *Cache-Estimated Significance* (CES) model for the estimation of top- k query results using cached information, where parameters such as k , the cached neighbor list size λ , and the cache size are all allowed to vary. The model includes shared-neighbor similarity measures that, given any object $w \in S$, assesses the statistical significance of the relationship between v and w using only the information available in the cache. Using one of these measures, an approximation to the top- k query result for v can be generated by determining the k objects of S most closely related to v .

4.1 Shared-Neighbor Similarity Measures

Before presenting the details of the CES model and associated similarity measures, we give a spatially-motivated argument concerning the potential of shared neighbor information for indicating similarity relationships among data objects, even when the underlying distance values are unknown.

Consider the situation in which the objects of a data set S are embedded in a metric space with distance function *dist*, from which the relevance ranking function Q is derived. Let x be any point in the space (not necessarily coincident with an object of S). With respect to any object $v \in S$, we can define *rank*(v, x) to be the rank that a new object would be assigned if it were inserted into S at location x , with any tied distances broken in favor of x . Point x would also determine a standard relevant set $Q(x, \lambda)$ and inverted relevant set $Q^{-1}(x, \lambda)$, with respect to S using *dist*.

If x were allowed to migrate towards the location of v , the memberships of $Q(x, \lambda)$ and $Q^{-1}(x, \lambda)$ would tend progressively toward those of $Q(v, \lambda)$ and $Q^{-1}(v, \lambda)$, respectively, until they coincided at $x = v$. The relationships between $Q(x, \lambda)$ and $Q(v, \lambda)$ on the one hand, and $Q^{-1}(x, \lambda)$ and $Q^{-1}(v, \lambda)$ on the other, can serve as the foundation of a *rank measure* of the similarity between x and v . Each object of $Q(x, \lambda) \cap Q(v, \lambda)$ would support the contention that x and v were similar, and each object of $Q(x, \lambda) \setminus Q(v, \lambda)$ or $Q(v, \lambda) \setminus Q(x, \lambda)$ would work against it. The same would be true for inverted sets, with each of $Q^{-1}(x, \lambda) \cap Q^{-1}(v, \lambda)$ testifying as to the similarity of x and v , and each object of $Q^{-1}(x, \lambda) \setminus Q^{-1}(v, \lambda)$ or $Q^{-1}(v, \lambda) \setminus Q^{-1}(x, \lambda)$ refuting it.

For the cache entry estimation problem, let us assume

that we wish to estimate the top- k relevant set $Q(v, k)$ for some object $v \notin C$. Even though only rank information is available and any spatial embedding and distance information is unknown, the spatial intuition described above can still be expected to apply in many (if not most) practical settings. The cached object set C would constitute a set of potential *witnesses* to the relationship between two objects v and w in S . However, as the cache generally contains only a small fraction of the total possible standard relevant sets, any rank measure of similarity must rely primarily on the inverted relevant sets. The set of witnesses $Q_C^{-1}(v, \lambda)$ would be non-empty provided that v appeared in at least one cached standard relevant set.

Just as different formulations of shared-neighbor criteria have been proposed for clustering applications, it is possible to devise many shared-neighbor similarity measures for our estimation problem. Given an inverted cache $C^{-1}(C, \lambda)$ and two inverted relevant sets $Q_C^{-1}(v, \lambda)$ and $Q_C^{-1}(w, \lambda)$, we focus on the following three cache-estimated rank measures:

- the *intersection size* measure

$$\text{SimInt}_{C,\lambda}(v, w) \triangleq |Q_C^{-1}(v, \lambda) \cap Q_C^{-1}(w, \lambda)|;$$

- the *set correlation* measure

$$\text{SimCorr}_{C,\lambda}(v, w) \triangleq R_C(Q_C^{-1}(v, \lambda), Q_C^{-1}(w, \lambda));$$

- and the *cosine* measure

$$\text{SimCos}_{C,\lambda}(v, w) \triangleq \text{CM}(Q_C^{-1}(v, \lambda), Q_C^{-1}(w, \lambda)).$$

The cosine measure is included here as a simplified alternative to the set correlation measure — in practical settings, the value of λ and the sizes of the witness sets $Q_C^{-1}(v, \lambda)$ and $Q_C^{-1}(w, \lambda)$ are very much smaller than S and C , and the difference between $\text{SimCos}_{C,\lambda}$ and $\text{SimCorr}_{C,\lambda}$ is negligible.

For all three measures, a value of 1 indicates that all occurrences of v in the cached top- k query result lists coincide with occurrences of w (and vice versa), and thus that the cache strongly supports the association of v and w . On the other hand, values approaching 0 indicate little support for the association of v and w . The set correlation and cosine measures are not well-defined whenever $Q_C^{-1}(v, \lambda) = \emptyset$ or $Q_C^{-1}(w, \lambda) = \emptyset$. For these cases, the values of the measures are taken to be 0. For the set correlation measure, note that $Q_C^{-1}(v, \lambda)$ and $Q_C^{-1}(w, \lambda)$ are subsets of C , and thus the characteristic vector descriptions take C to be the universal set, and not S .

4.2 Significance of Similarity Measures

The rank-based similarity measures $\text{SimInt}_{C,\lambda}$, $\text{SimCos}_{C,\lambda}$ and $\text{SimCorr}_{C,\lambda}$ stated above may have different biases with respect to the cache size $|C|$, the relevant set size λ , and the sizes of the inverted relevant sets involved. The intersection size measure $\text{SimInt}_{C,\lambda}$ potentially favors those objects w having the largest witness sets $Q_C^{-1}(w, \lambda)$, as no penalty is applied when members of this set do not appear in $Q_C^{-1}(v, \lambda)$. The other two rank-based measures compensate for this deficiency by normalizing the contributions with respect to the sizes of the witness sets. However, these two measures also admit the possibility of bias. In general, when making inferences involving Pearson correlation, a high correlation value alone is not considered sufficient to judge the significance of the relationship between two variables. When

the number of variable pairs is small, it is much easier to achieve a high value by chance than when the number of pairs is large.

The RSC model for clustering was proposed as a way of correcting for the bias in shared-neighbor density measures of cluster quality [9]. The statistical significance of formulas involving set correlation values was tested against a ‘hypothesis of randomness’ — the assumption that each relevant set contributing to the density measure is independently generated via uniform random selection from among the available objects of S . In practice, of course, the relevant sets are far from random. However, the hypothesis serves as an appropriate reference point from which the significance of observed values of the measure can be assessed. Under the randomness hypothesis, the mean and standard deviation of the measure can be calculated, and standard scores (also known as Z -scores) [19] can then be generated and compared with one another. The more significant grouping would be the one whose standard score is highest — that is, the one whose correlation exceeds its expected value by the greatest number of standard deviations.

For the proposed cache entry estimation model of this paper, we will use the hypothesis of randomness to account for potential bias of $\text{SimInt}_{C,\lambda}$ and $\text{SimCorr}_{C,\lambda}$ due to the choice of standard cache list size λ , and the variations in the size of the inverted relevant sets that accompany the choice of λ . Here, we will test against the hypothesis that the relevant sets have been generated by random selection of objects, with each object selected independently with fixed probability.

LEMMA 1. *Let A be a set generated through independent random selection from the objects of set Ω , with each object present in A with probability $0 < p < 1$. Let B be a second set generated independently from Ω in the same manner as A . Then*

- $\mathbf{E}[|A \cap B|] = p^2 \cdot |\Omega|$ and $\mathbf{Var}[|A \cap B|] = p^2(1 - p^2) \cdot |\Omega|$,
- $\mathbf{E}[R_\Omega(A, B)] = 0$ and $\mathbf{Var}[R_\Omega(A, B)] = \frac{1}{|\Omega| - 1}$.

PROOF. Omitted in this version due to space limitations. $\square$

Let $\text{SimInt}_{C,\lambda}(v, w)$ be a random variable representing the value of $\text{SimInt}_{C,\lambda}(v, w)$ that would be achieved if all cached standard relevant sets were independently generated via the uniform random selection of λ objects from the data set S . Similarly, let $\text{SimCorr}_{C,\lambda}(v, w)$ be a random variable representing the value of $\text{SimCorr}_{C,\lambda}(v, w)$ that would be achieved under the same conditions. The probability p of an individual object $u \in C$ appearing in $Q_C^{-1}(v, \lambda)$ would equal that of v appearing in $Q(u, \lambda)$, which is $p = \frac{\lambda}{|S|}$. Lemma 1 can then be applied with $\Omega = C$, $A = Q_C^{-1}(v, \lambda)$, $B = Q_C^{-1}(w, \lambda)$, and $p = \frac{\lambda}{|S|}$ to show that

- $\mathbf{E}[\text{SimInt}_{C,\lambda}(v, w)] = |C| \frac{\lambda^2}{|S|^2}$ and $\mathbf{Var}[\text{SimInt}_{C,\lambda}(v, w)] = |C| \frac{\lambda^2}{|S|^2} \left(1 - \frac{\lambda^2}{|S|^2}\right)$, and
- $\mathbf{E}[\text{SimCorr}_{C,\lambda}(v, w)] = 0$ and $\mathbf{Var}[\text{SimCorr}_{C,\lambda}(v, w)] = \frac{1}{|C| - 1}$.

Given a set of cache objects C , the *cache-estimated correlation significance* of w relative to query v is defined as the

standard score for $SimCorr_{C,\lambda}(v, w)$ under the randomness hypothesis:

$$\begin{aligned}
ZCorr_{C,\lambda}(v, w) &\triangleq \frac{SimCorr_{C,\lambda}(v, w) - \mathbf{E}[SimCorr_{C,\lambda}(v, w)]}{\sqrt{\mathbf{Var}[SimCorr_{C,\lambda}(v, w)]}} \\
&= \sqrt{|C| - 1} SimCorr_{C,\lambda}(v, w) \\
&= \sqrt{|C| - 1} \frac{|C|}{\sqrt{(|C| - |A|)(|C| - |B|)}} \cdot \left(\frac{|A \cap B|}{\sqrt{|A| \cdot |B|}} - \frac{\sqrt{|A| \cdot |B|}}{|C|} \right),
\end{aligned}$$

where $A = Q_C^{-1}(v, \lambda)$ and $B = Q_C^{-1}(w, \lambda)$, and $|A \cap B| = SimInt_{C,\lambda}(v, w)$. This indicates that as long as the cache size $|C|$ is kept constant, the correlation significance of w relative to v is equivalent to the correlation itself for the purposes of ranking.

The *cache-estimated intersection significance* of w relative to query v is defined as the standard score for $SimInt_{C,\lambda}(v, w)$ under the randomness hypothesis:

$$\begin{aligned}
ZInt_{C,\lambda}(v, w) &\triangleq \frac{SimInt_{C,\lambda}(v, w) - \mathbf{E}[SimInt_{C,\lambda}(v, w)]}{\sqrt{\mathbf{Var}[SimInt_{C,\lambda}(v, w)]}} \\
&= \frac{SimInt_{C,\lambda}(v, w) - np^2}{p\sqrt{n(1 - p^2)}} \\
&= \sqrt{|C|} \frac{|S|}{\sqrt{|S|^2 - \lambda^2}} \left(\frac{SimInt_{C,\lambda}(v, w)}{\frac{\lambda \cdot |C|}{|S|}} - \frac{\lambda}{|S|} \right).
\end{aligned}$$

4.3 Ranking Functions

The use of standard scores as a measure of statistical significance can facilitate comparison across differing distributions. The cache-estimated significance measures, being standard scores, can thus be used to account for significance across such parameter choices as the data set size $|S|$, the number of cache entries $|C|$, the cached list size λ , the query result size k , and the sizes of inverted sets. At query time, the parameters S and C can be considered to be fixed quantities, and the sizes of inverted sets vary according to the distribution of objects in the vicinity of the query object.

Under certain conditions, some of the secondary similarity measures proposed in this section turn out to be equivalent. Cache and data set sizes can greatly exceed the maximum sizes of the standard and inverted relevant sets; in these situations, the value of the set correlation measure $SimCorr_{C,\lambda}$ tends to that of the cosine measure $SimCos_{C,\lambda}$. When the cache size is considered to be fixed, $ZCorr_{C,\lambda}$ and $SimCorr_{C,\lambda}$ determine the same rankings of data set objects, and can be used interchangeably. The latter measure is in fact more convenient to use as its values are restricted to the range $[-1, 1]$. If the relevant set size λ is also taken to be fixed, the measure $ZInt_{C,\lambda}$ determines the same ranking of data objects as $SimInt_{C,\lambda}$. Similarly, the rankings due to $ZInt_{C,\lambda}(v, w)$ would be the same as those determined by the ratio between the inverted set intersection size and the average individual inverted set size:

$$SimRatio_{C,\lambda}(v, w) \triangleq \frac{SimInt_{C,\lambda}(v, w)}{\lambda \cdot |C|/|S|}.$$

However, in general, neither of these equivalences hold if λ is allowed to vary. It should be noted that unlike $SimInt_{C,\lambda}$, $SimCos_{C,\lambda}$ and $SimCorr_{C,\lambda}$, the measure $SimRatio_{C,\lambda}$ may attain values exceeding 1.

If the values of $SimInt_{C,j}(v, w)$ are readily available for all $1 \leq j \leq \lambda$, each object w can be scored with respect to query v according to the query size j for which it is most significant. Treating both the data set size $|S|$ and cache size $|C|$ as constants, for the experimental evaluation to follow we shall consider the following six ranking functions for the objects of S with respect to query object v :

- (derived from the significance of the intersection size measure $SimInt$) the ratio measure $SimRatio_{C,\lambda}(v, w)$ and the *max-ratio* measure

$$MaxRatio_{C,\lambda}(v, w) \triangleq \max_{1 \leq j \leq \lambda} SimRatio_{C,j}(v, w);$$

- (derived from the significance of the set correlation measure $SimCorr$) the set correlation measure $SimCorr_{C,\lambda}(v, w)$, and the *max-correlation* measure

$$MaxCorr_{C,\lambda}(v, w) \triangleq \max_{1 \leq j \leq \lambda} SimCorr_{C,j}(v, w);$$

- (derived from the limit of the set correlation measure $SimCorr$ as the database and cache sizes increase) the cosine measure $SimCos_{C,\lambda}(v, w)$, and the *max-cosine* measure

$$MaxCos_{C,\lambda}(v, w) \triangleq \max_{1 \leq j \leq \lambda} SimCos_{C,j}(v, w).$$

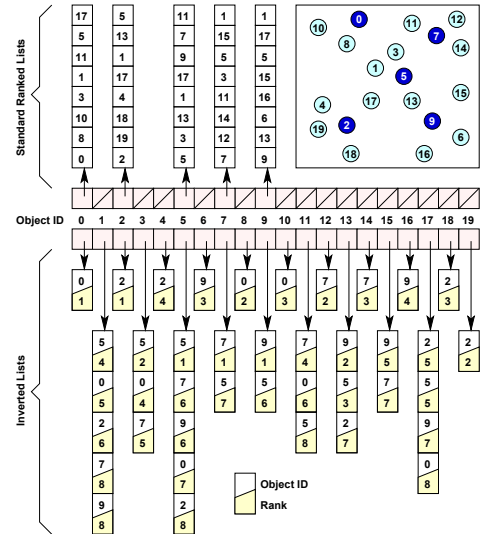


Figure 1: Cache data structures for a set of 20 objects in the 2-D plane. Top-8 lists are cached for 5 objects, with the Euclidean distance as the underlying ranking function.

5. IMPLEMENTATION

When standard cache $\mathcal{C}(C, \lambda)$, its inverted cache $\mathcal{C}^{-1}(C, \lambda)$, and their subcaches are all available in main memory, the intersection sizes $SimInt_{C,j}(v, w)$ can be efficiently calculated for all $1 \leq j \leq \lambda$. A practical assumption that lowers the computational cost is that w need only be evaluated if the intersection size $SimInt_{C,j}(v, w)$ is positive — otherwise, there

Euclidean		SimCorr		MaxCorr		SimCos		MaxCos		SimRatio		MaxRatio		
w	dist	w	score	w	score	j	w	score	j	w	score	w	score	j
11	0.000	11	1.000	11	1.000	4	11	1.000	4	11	1.500	11	1.500	8
7	1.000	3	1.000	3	1.000	8	3	1.000	8	1	1.500	1	1.500	8
3	1.202	7	0.667	7	1.000	4	7	0.817	7	1.000	4	3	1.500	8
12	1.647	0	0.408	12	1.000	4	1	0.775	12	1.000	4	5	1.500	8
0	1.944	8	0.408	14	1.000	4	5	0.775	14	1.000	4	7	1.000	4
5	1.961	10	0.408	0	0.612	6	0	0.577	1	0.775	8	17	1.000	12
14	2.000	12	0.408	8	0.612	6	8	0.577	5	0.775	8	0	0.500	14
1	2.119	14	0.408	10	0.612	6	10	0.577	0	0.707	6	8	0.500	17
8	2.499	1*	undef	5	0.408	7	12	0.577	8	0.707	6	9	0.500	0
13	2.832	5*	undef	15	0.167	7	14	0.577	10	0.707	6	10	0.500	8
15	3.102	9*	-0.167	1*	-0.167	6	17	0.577	17	0.577	8	12	0.500	10
17	3.201	15*	-0.167	2*	-0.250	4	9	0.408	15	0.500	7	13	0.500	15
10	3.432	17*	-0.408	4*	-0.250	4	15	0.408	9	0.408	8	14	0.500	9
9	3.655	2*	-0.612	6*	-0.250	4	13	0.333	13	0.333	8	15	0.500	13
2	4.425	4*	-0.612	9*	-0.250	4	2*	0.000	2*	0.000	4	2*	0.000	2*
4	4.432	6*	-0.612	16*	-0.250	4	4*	0.000	4*	0.000	4	4*	0.000	4
6	4.541	16*	-0.612	18*	-0.250	4	6*	0.000	6*	0.000	4	6*	0.000	4
16	4.791	18*	-0.612	19*	-0.250	4	16*	0.000	16*	0.000	4	16*	0.000	4
19	5.175	19*	-0.612	13*	-0.408	4	18*	0.000	18*	0.000	4	18*	0.000	4
18	5.268	13*	-0.667	17*	-0.408	5	19*	0.000	19*	0.000	4	19*	0.000	4

Figure 2: Actual and estimated rankings of the neighbors of object 11 from the example of Figure 1, with $\lambda = 8$, $|C| = 5$, and $|S| = 20$. Euclidean distances are expressed as a proportion of the distance between object 11 and its nearest neighbor, object 7. For *MaxCorr*, *MaxCos* and *MaxRatio*, the maximum is achieved at the stated relevant-set size j . Those objects whose low measure values preclude them from appearing in the query result are indicated by an asterisk.

is no information supporting the contention that w should be included in the estimated query result for v . Moreover, even if the intersection size is positive, a negative value of the correlation measure *MaxCorr* (or *SimCorr*) would indicate that the intersection between the inverted neighborhoods is less than what would be expected if the members of the original standard neighborhoods had been selected at random. Therefore, for any ranking function f chosen from among the six listed in the previous section, we assume that a threshold value $\varphi > 0$ has been supplied, and that only those $w \in S$ for which $f(v, w) \geq \varphi$ are eligible to appear in the estimated query result for v . If it turns out that fewer than the requested k objects are eligible, then the estimation is deemed to have failed, possibly necessitating an exact computation of the query result from information residing on disk. An example is provided in Figure 2.

The efficiency of the ranking process also depends on the storage of additional information with the witness sets:

1. For all $v \in C$, with each object $u \in Q_C^{-1}(v, \lambda)$, the rank $rank(u, v)$ of v in $Q(u, \lambda)$ is stored.
2. The objects of $Q_C^{-1}(v, \lambda)$ are listed in non-decreasing order of these stored rank values.

With these preparations, the objects of set $Q_C^{-1}(v, j)$ are simply those objects $u \in Q_C^{-1}(v, \lambda)$ with stored rank value $rank(u, v) \leq j$, which can be read off from the head of the list in time proportional to the size of $Q_C^{-1}(v, j)$.

The proposed method for the cache-estimated ranking of all objects of S with respect to a query object v is summarized in Figure 12. In the pseudocode description, all objects are represented by IDs in the range $[0, \dots, |S|]$. For any object $u \in C$, we denote by $q(u, j)$ the ID of the object of rank j in the ranked list $Q(u, \lambda)$. Also, we denote by $q^{-1}(v, j) = \{u \in C \mid v \in Q(u, \lambda) \wedge rank(u, v) = j\}$ the set of IDs of objects of $Q_C^{-1}(v, \lambda)$ having rank j . Note that the objects IDs of $Q_C^{-1}(v, \lambda)$ are assumed to be sorted in terms of these ranks (as illustrated in Figure 1), and thus the objects of $q^{-1}(v, j+1)$ appear immediately after those of $q^{-1}(v, j)$ in $Q_C^{-1}(v, \lambda)$.

In order to compute meaningful query result estimates, the cache size should be chosen so as to ensure that the inverted relevant sets are sufficiently large. This implies that $|C|$ and λ should be chosen so that the average inverted list size $\frac{\lambda \cdot |C|}{|S|}$ — the number of witnesses of the relationship be-

tween the query and its estimated result objects — exceeds some supplied threshold $\tau > 0$. Since $\lambda \cdot |C| \geq \tau \cdot |S|$, the main memory must be sufficiently large to be able to accommodate at least a constant amount of storage for every object in the data set.

Computing the sizes of the intersections of witness sets $SimInt_{C,j}(v, w)$ for every possible $w \in S$, if performed in the most straightforward manner, would be too costly an operation even with all necessary information resident in main memory. The computational cost is reduced by first observing that for the majority of points $w \in S$, the intersection between $Q_C^{-1}(v, j)$ and $Q_C^{-1}(w, j)$ is empty. As it is reasonable to assume that only positive values of $SimInt_{C,j}$ are meaningful, we limit the ranking effort to those w for which $Q_C^{-1}(v, j) \cap Q_C^{-1}(w, j) \neq \emptyset$. These objects are precisely those of the set $N(v, j) \triangleq \bigcup_{u \in Q_C^{-1}(v, j)} Q(u, j)$, which could be constructed in time proportional to $O(j \cdot |Q_C^{-1}(v, j)|)$, which is approximately $\tilde{O}(j^2)$. However, the computation time is further reduced by computing the intersection sizes $SimInt_{C,j}(v, w)$ incrementally during the visitation of the members of $N(v, j)$, by noting that

$$|Q_C^{-1}(v, j) \cap Q_C^{-1}(w, j)| = |\{u \in Q_C^{-1}(v, j) : w \in Q(u, j)\}|.$$

The overall cost of ranking all objects is thus reduced to $O(j \cdot |Q_C^{-1}(v, j)| + |N(v, j)|)$, or approximately $\tilde{O}(j^2)$.

The value $SimInt_{C,j+1}(v, w)$ is calculated as the sum of $|[Q_C^{-1}(v, j+1) \setminus Q_C^{-1}(v, j)] \cap Q_C^{-1}(w, j+1)|$ and $|Q_C^{-1}(v, j) \cap [Q_C^{-1}(w, j+1) \setminus Q_C^{-1}(w, j)]|$ with $SimInt_{C,j}(v, w)$; only the first two expressions need be explicitly computed in the transition from j to $j+1$. The total time, therefore, for computing $SimInt_{C,j}(v, w)$ for all $1 \leq j \leq \lambda$ is simply that of computing $SimInt_{C,\lambda}(v, w)$, which as mentioned earlier is $O(\lambda \cdot |Q_C^{-1}(v, \lambda)| + |N(v, \lambda)|)$, or approximately $\tilde{O}(\lambda^2)$.

6. EVALUATION

6.1 Performance Measures and Data Sets

As already discussed in Section 2, existing active caching methods developed for Boolean queries cannot in general be applied to handle similarity queries, and thus a direct comparison between these methods and our proposed techniques is not possible. For this reason, to evaluate the active caching strategies for top- k similarity queries proposed

in Section 4, we instead compare their performance against those of passive caching strategies in terms of two measures, the *hit rate* and the *recall*. Traditionally, given a schedule of queries, the hit rate of an active caching method is defined to be the proportion of queries for which the sought-after information resides in the cache. Here, we extend the definition to include those cases where a query result can be estimated using our active caching methods — for all of the proposed methods, an estimate can be returned whenever a non-empty cache inverted list is associated with the query item. A miss occurs when a query item is associated with neither a standard list nor an inverted list in the cache.

Consider now the item set retrieved by any given top- k query operating on the cache. The recall of the query is defined as the proportion of this result that would also appear in a top- k query applied to the full database. When the cache query and the database query have equal size k , this definition is equivalent to that of query precision. Note that when the top- k list is explicitly stored in the cache, the recall trivially equals 1, and when a cache miss occurs, the recall is 0. We will also be interested in assessing the overall estimation power of the proposed active caching methods. To do this, we report the recall that would be achieved for the query result estimated by the active caching method, without checking whether the true query result is explicitly stored in the cache. To distinguish the two interpretations of recall, we shall refer to this latter interpretation as the *estimation recall*, and the former (usual) interpretation as the *cache recall*.

Four data sets were used in our experimentation: the ALOI data set (ALOI) [7] consisting of 110,250 images represented as 641-dimensional feature vectors [2]; the Reuters Corpus Volume 1 (RCV1) news article data set [15] represented as 802,352 document vectors; the Cover Type (CT) [1] set taken from the UCI Machine Learning Repository, containing 581,012 observations of forest cover type from U.S. governmental cartographic surveys; and a sample of 120836 instances from the KDD Cup 1999 (KDD) data set [1], each corresponding to a simulated intrusion event in a military network environment. For each set, 100-NN lists were computed using the SASH approximate similarity search structure [10]. Euclidean distances were computed for each set except RCV1, for which vector angle distances were used.

6.2 Experimental Results

For both the ALOI and RCV1 data sets, we conducted several k -nearest-neighbor retrieval experiments, over a variety of cache and list sizes, and of choices of k . For each experiment, subsets of the data objects were selected uniformly at random for inclusion in the standard cache, at different proportions of the total data set size. The cached information consisted of the standard neighbor list for each cache item, and their associated inverted lists. After setting up the cache, each of the objects of the full data set was used as the basis of a k -nearest-neighbor query. In all the experiments that follow, experimental results are shown only for the *SimCorr*, *MaxCorr*, *SimRatio* and *MaxRatio* methods, as the values *SimCos* and *MaxCos* were virtually identical to those of *SimCorr* and *MaxCorr* for even the smallest of the cache sizes considered.

6.2.1 Efficiency

For the ALOI and RCV1 data sets, Figure 3 shows the

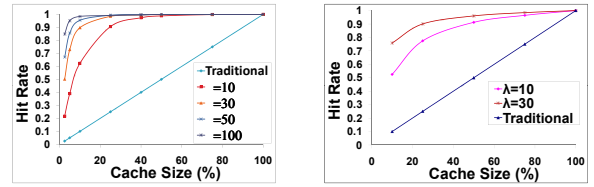


Figure 3: Hit rate for active caching for ALOI (left) and RCV1 (right), plotted against the proportion of objects cached.

hit rate achieved by the active caching strategy for different choices of the standard list size λ . The proportion of items cached for this experiment varied between 2.5% and 100%. In all cases, the hit rates were much higher with active caching than for passive caching, increasing very quickly with increasing list size. When the space required by the inverted lists is taken into account, the hit rates for all four data sets studied still greatly exceed those of traditional caching, as can be seen in Figure 4.

The CES solution can require more space than the traditional caching approach due to the storage of inverted lists alongside standard result lists. For example, in a straightforward implementation in which integer variables are stored using 4 bytes, each result item would be associated with up to 12 bytes of storage: an integer object ID in the standard list, and an integer object ID plus (for *MaxCorr* and *MaxRatio*) an integer rank in the corresponding inverted list entry. If traditional caching is performed with only object IDs being stored, only 4 bytes would be required for each result list entry, and thus active caching would require approximately 3 times as much storage as the traditional implementation. Nevertheless, it can be seen from Figure 4 that for cache sizes of approximately less than one-third of the data, our active caching solutions can answer substantially more queries than traditional caching even taking its potentially-increased memory requirements into account.

6.2.2 Effectiveness

Figure 5 shows the cache recall values obtained over active caching hits for top-30 queries with standard list length $\lambda = 30$. For all four data sets, the CES solutions achieved very high recall values across the range of cache sizes. Together with Figures 3 and 4, these results show that for sufficiently-large cache sizes, very effective recall rates can be achieved while only rarely needing to access information on disk. For example, for the ALOI set with 25% of the data items cached, the proposed active caching variants answered approximately 98% of the queries, with an average recall of 0.8 and above. Even for the smallest cache size studied, 50% of the queries were answered with recall rates above 0.65, whereas the traditional caching approach would answer only 2.5% of the queries (albeit with recall 1.0).

Next, a similar experiment was performed on the ALOI data set for one representative measure, *SimRatio*, this time varying the size of the query result k together with the cached standard list size λ . Top- k queries were performed with $k = \lambda = 10, 30, 50$ and 100. The results, shown in Figure 6, did not vary significantly for different settings of λ ; however, the same general dependence on the cache size was observed as in the previous experiment.

For the remainder of the experiments in this section, we concentrate on the estimation power of our active caching

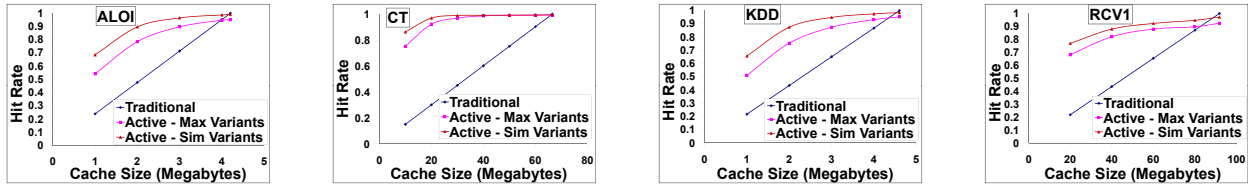


Figure 4: Hit rate in Mb for ALOI, RCV1, CT, and KDD, for the case $k = \lambda = 10$. Traditional caching assumes $4\lambda = 40$ bytes per object cached, whereas active caching requires $8\lambda = 80$ bytes (*SimCorr* and *MaxRatio*) or $12\lambda = 120$ bytes (*MaxCorr* and *MaxRatio*).

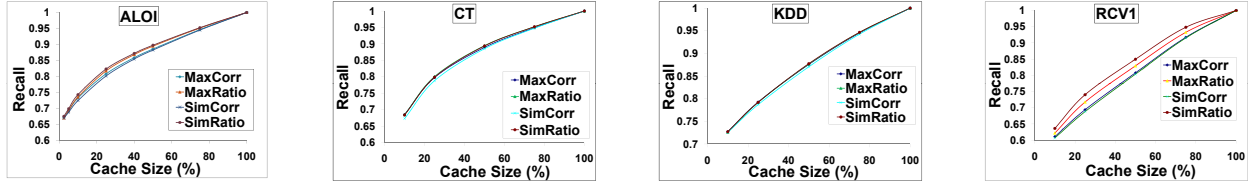


Figure 5: Average recall with $k = \lambda = 30$, plotted against the proportion of objects cached.

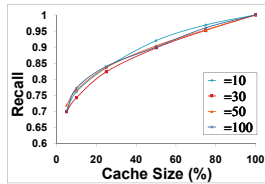


Figure 6: Average ALOI cache recall for top- k active caching hits using the *SimRatio* measure, with list sizes $\lambda = k$, plotted against the proportion of objects cached.

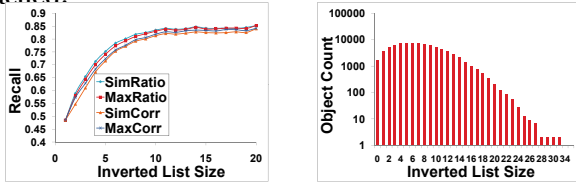


Figure 7: Average estimation recall values for active cache hits as a function of query inverted list size, for the ALOI data set with $k = \lambda = 30$ and 25% of objects present in the cache. The higher list size values are omitted due to the small number of instances. The histogram shows the numbers of query items with inverted lists of a given size.

methods, by forcing an active estimation in which the standard list stored for the item at which the query is based is always ignored.

We conducted an experiment to show the relationship between observed average estimation recall rates and the sizes of the witness lists associated with query items. In Figure 7 the average recall values for the ALOI data set are plotted as a function of query witness list size, together with a histogram showing the numbers of query items involved. Top-30 queries were performed based at all 110,250 ALOI images. 25% of the ALOI images were selected at random for inclusion in the cache, each having a standard list of size $\lambda = 30$. The experimental results show consistently-high average estimation recall rates over all but the smallest witness list sizes, with *MaxRatio* and *SimRatio* performing slightly better than *MaxCorr* and *SimCorr* over most of the range. They suggest that better performances are achieved for larger query witness list sizes.

We next considered the effect on estimation recall when

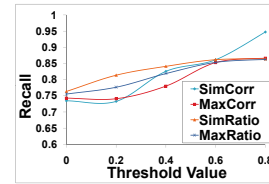


Figure 8: Average estimation recall rates of top-30 active cache hits for the ALOI data set, in which all items in the result list satisfy minimum thresholds on similarity measure values.

minimum thresholds are applied to *SimRatio*, *MaxRatio*, *SimCorr* and *MaxCorr* scores. Again, top-30 queries were performed for the ALOI data set, with 25% of the images cached and $\lambda = 30$. In this experiment, active caching was used to generate a result only when each item in the estimated top- k result list achieved a score higher than a specified minimum threshold value. Figure 8 shows plots of the average recall for the four active caching methods, against a range of score threshold values. The results show a positive influence between the minimum scores obtained and the recall rates achieved.

For each of the k -nearest-neighbor query experiments presented above, the standard list size λ was set equal to k . In order to provide insights into the effect of λ and k on the performance of the proposed active caching methods, a further experiment was conducted in which λ was varied while fixing k . Once again, a cache size of 25% was chosen from the ALOI data set, and k chosen to be 30. The results of the experiment are displayed in Figure 9. All of the caching methods performed best for the case $\lambda = k = 30$. For smaller values of λ , the performance degenerated markedly. For larger values, the recall rates remained high, although the *SimRatio* and *SimCorr* performances showed substantially more degradation than those of *MaxRatio* and *MaxCorr*. Although *SimRatio* performed marginally better than *MaxRatio* for the case $\lambda = k = 30$, the latter method achieved the best overall performance when larger list sizes were used.

6.2.3 Robustness

We tested our proposed solution for robustness by introducing noise entries into the cached standard lists. In each test, ALOI cache lists were replaced by “noise lists” of the

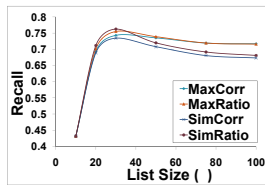


Figure 9: The effect of varying cache list size λ on the average estimation recall rates of top-30 similarity query cache hits, for the ALOI data set with 25% of the objects cached.

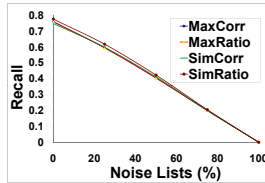


Figure 10: Average estimation recall rates for ALOI active cache hits, with 25% of the objects cached and with $k = \lambda = 30$, plotted against the proportion of noise lists.

same length, generated by selecting objects from the full data set uniformly at random. The proportion of cache lists replaced by noise lists was varied between 0% and 100%. The experimental results in Figure 10 show a very strong linear relationship between the performance and the proportion of noise. The results indicate that relatively large amounts of noise can be tolerated while still providing very high recall rates.

It is possible that for some caching applications, the stored result lists may not all be of the same length, as has been assumed so far in our experimentation. Accordingly, we conducted an experiment on the ALOI data set in which top-30 query result estimation was performed, with a varying proportion of the standard cache list lengths selected uniformly at random between 1 and 100, and the remainder of the lists having lengths fixed at 30. The cache size was chosen to be 25%. The results of the experiment are shown in Figure 11. Although some degradation of performance was observed for the *SimCorr* and *SimRatio* measures, the estimation recall rates of *MaxCorr* and *MaxRatio* were remarkably stable across the full range of proportions. The results confirm the ability of the CES model to correct for bias with respect to the sizes of standard lists, in particular for those variants in which the significance is maximized over a range of subcache sizes.

A final experiment was conducted to determine the influence of the choice of k on the recall rate of top- k active cache queries, when the lengths of the cache standard lists is variable. In this test, the list lengths were selected uniformly at random in the range 1 to 100. The cache size was again chosen to be 25%. Figure 11 shows a peak in performance for all methods for k between 30 and 40. When the standard list lengths differ greatly from k , the performance degrades due to the presence of irrelevant items in cached lists (when $\lambda > k$) or the absence of relevant items (when $\lambda < k$). For this experiment, with list lengths ranging between 1 and 100, choices of k in the range 30 to 40 achieved the best tradeoff between these two sources of error.

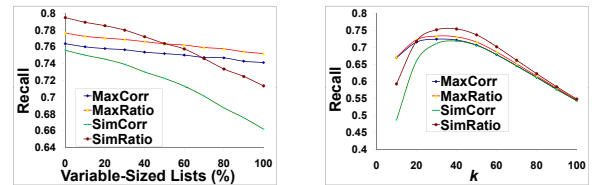


Figure 11: Average estimation recall rates of active cache hits for the ALOI data set, with 25% of the objects cached. The left plot shows the recall for top-30 queries against various proportions of standard lists with length λ ranging between 1 and 100, with the remainder of the lists having length 30. The right plot shows recall for different values of k with all standard lists having lengths randomly selected between 1 and 100.

7. CONCLUSION

The paper proposed a general model, the Cache-Estimated Significance (CES), for the estimation of the results of top- k similarity queries using shared-neighbor similarity measures on cached information. The model does not assume any knowledge of the methods or similarity measures used, and as such can be applied even when non-metric and probabilistic approaches are used to produce query results.

The main contribution of the CES approach is to facilitate the design of shared-neighbor ranking formulae for active caching that allow for variation of (and comparison across) such parameters as the size of the cache, the length of ranked lists stored in the cache, and the number of items requested by the query. The experimental results of the previous section indicate that the performance of the CES-derived *MaxRatio* and *MaxCorr* ranking functions is significantly more stable than their counterparts *SimRatio* and *SimCorr* when cached list sizes were allowed to vary.

The very strong overall performance of the shared-neighbor ranking formulae for active caching provides an intriguing answer to the question of cache management for databases. Whereas the conventional approach is to fill the cache with those items most likely to be requested in future queries, our experimental results show that shared-neighbor active caching can instead support a form of *data interpolation*, in which the cache is selected so as to provide uniform coverage of the data set from which most if not all query results are actively generated. For some applications, it may even suffice to answer all similarity queries actively, without ever referring to the original data. CES active caching could thus serve as a scalability technique, as it provides the basis of space- and time-efficient approximation of large databases.

8. REFERENCES

- [1] A. Asuncion and D. J. Newman, *UCI Machine Learning Repository*, Irvine, CA: University of California, School of Information and Computer Science, 2007. [<http://www.ics.uci.edu/ml/learn/MLRepository.html>]
- [2] N. Boujemaa, J. Fauqueur, M. Ferecatu, F. Fleuret, V. Gouet, B. Le Saux and H. Sahbi, "IKONA: Interactive Generic and Specific Image Retrieval," in *Proc. Intern. Workshop on Multimedia Content-Based Indexing and Retrieval (MMCBIR)*, Rocquencourt, France, 2001.
- [3] B. Chidlovskii and U. Borghoff, "Semantic caching of web queries," *The VLDB Journal*, vol. 9, no. 1, pp. 2–17, 2000.
- [4] S. Dar, M. J. Franklin, B. Jónsson, D. Srivastava and M. Tan, "Semantic data caching and replacement," in *Proc.*

- 22nd Intern. Conf. on Very Large Data Bases (VLDB), San Francisco, CA, USA, 1996, pp. 330–341.
- [5] L. Ertöz, M. Steinbach and V. Kumar, “Finding clusters of different sizes, shapes, and densities in noisy, high dimensional data,” in *Proc. 3rd SIAM Intern. Conf. on Data Mining (SDM)*, San Francisco, USA, 2003, pp. 47–58.
- [6] M. Ester, H.-P. Kriegel, J. Sander and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proc. 2nd Int. Conf. on Knowl. Discovery and Data Mining (KDD)*, Portland, OR, USA, 1996, pp. 226–231.
- [7] J. M. Geusebroek, G. J. Burghouts and A. W. M. Smeulders, “The Amsterdam library of object images,” *Int. J. Comput. Vision*, vol. 61, pp. 103–112, 2005.
- [8] S. Guha, R. Rastogi and K. Shim, “ROCK: a robust clustering algorithm for categorical attributes,” *Inform. Sys.*, vol. 25, pp. 345–366, 2000.
- [9] M. E. Houle, “The relevant-set correlation model for data clustering,” *Statistical Analysis and Data Mining*, vol. 1, no. 3, pp. 157–176, 2008.
- [10] M. E. Houle and J. Sakuma, “Fast approximate similarity search in extremely high-dimensional data sets,” in *Proc. 21st IEEE Int. Conf. on Data Eng. (ICDE)*, Tokyo, Japan, 2005, pp. 619–630.
- [11] R. A. Jarvis and E. A. Patrick, “Clustering using a similarity measure based on shared nearest neighbors,” *IEEE Trans. Comput.*, vol. C-22, pp. 1025–1034, 1973.
- [12] M. G. Kendall and J. D. Gibbons, *Rank Correlation Methods*, 5th ed., Charles Griffin, 1990.
- [13] T. M. Kroege, D. D. E. Long and J. C. Mogul, “Exploring the bounds of web latency reduction from caching and prefetching,” in *Proc. USENIX Symp. Internet Technologies and Systems*, December 1997.
- [14] K. C. K. Lee, H. V. Leong and A. Si, “Semantic query caching in a mobile environment,” *Mobile Computing and Comm. Rev.*, vol. 3, no. 2, pp. 28–36, Apr. 1999.
- [15] D. Lewis, Y. Yang, T. Rose and F. Li, “A new benchmark collection for text categorization research,” *J. Machine Learning Research*, vol. 5, pp. 361–397, 2004.
- [16] X. Long and T. Suel, “Three-level caching for efficient query processing in large web search engines,” in *Proc. 14th Int. Conf. on World Wide Web (WWW)*, Chiba, Japan, 2005, pp. 257–266.
- [17] Q. Luo, J. F. Naughton, and W. Xue, “Form-based proxy caching for database-backed web sites: keywords and functions,” *The VLDB Journal*, vol. 17, no. 3, May 2008, pp. 489–513.
- [18] E. Markatos and C. Chronaki, “A top-10 approach to prefetching the web,” in *Proc. 8th Ann. Conf. Internet Society (INET 1998)*, Geneva, Switzerland, 1998.
- [19] D. Moore, *Basic Practice of Statistics*, 4th ed., New York, USA: W.H. Freeman Company, 2006.
- [20] E. W. Pugh, L. R. Johnson and J. H. Palmer, *IBM’s 360 and Early 370 Systems (History of Computing)*, The MIT Press, 1991.
- [21] Q. Ren and M. H. Dunham, *Semantic Caching and Query Processing*, Southern Methodist University Technical Report TR-98-CSE-04, 1998.
- [22] G. Salton, *The SMART Retrieval System — Experiments in Automatic Document Processing*, Prentice-Hall, Englewood Cliffs, NJ, USA, 1971.
- [23] P. C. Saraiva, E. Silva de Moura, N. Ziviani, W. Meira, R. Fonseca and B. Riberio-Neto, “Rank-preserving two-level caching for scalable search engines,” in *Proc. 24th ACM SIGIR Conf. on Research and Development in Information Retrieval*, New Orleans, LA, USA, 2001.
- [24] S. M. Shafi and R. A. Rather, “Precision and recall of five search engines for retrieval of scholarly information in the field of biotechnology,” *Webology*, vol. 2, no. 2, Aug. 2005.
- [25] J. Zhang, X. Long and T. Suel, “Performance of compressed inverted list caching in search engines,” in

Proc. 17th Int. Conf. on World Wide Web (WWW), Beijing, China, April 2008, pp. 387–396.

- [26] B. Zheng, W.-C. Lee and D. L. Lee, “On semantic caching and query scheduling for mobile nearest-neighbor search,” *Wireless Networks*, vol. 10, no. 6, pp. 653–664, 2004.

Query

Input: query object ID v ;

Output: list *Result* of tuples (object ID, similarity value).

0. Initialization:

- Static integer array for storing the number of visits to objects, initialized to $RevIntCount[s] = 0$ for all object IDs $0 \leq s < |S|$.
- Static integer array for storing object visit flags, assumed set to $VisitFlag[s] = 0$ for all object IDs $0 \leq s < |S|$.
- Static real array storing object-to-query similarities $SimValue[s]$ for all object IDs $0 \leq s < |S|$.
- Initialize lists $Visited \leftarrow \emptyset$ and $NewlyVisited \leftarrow \emptyset$.

1. Set list $L \leftarrow Q_C^{-1}(v, \lambda)$ to contain the inverted relevant set for v , in the form of object IDs, with the entries sorted in non-decreasing order of the ranks they occupy in their neighbors’ λ -relevant sets.

2. For all $1 \leq j \leq \lambda$ do:

- Extract objects $L_j \leftarrow q^{-1}(v, j)$ from head of L .
- For all objects $u \in \bigcup_{i=1}^{j-1} L_i$ do:
 - Let $w \leftarrow q(u, j)$.
 - If $RevIntCount[w] = 0$, then w has been visited for the first time overall. Perform the list insertion $Visited \leftarrow Visited \cup \{w\}$.
 - Increment $RevIntCount[w] \leftarrow RevIntCount[w] + 1$.
 - If $VisitFlag[w] = 0$, then w has been visited for the first time in iteration j . Perform the list insertion $NewlyVisited \leftarrow NewlyVisited \cup \{w\}$, and set $VisitFlag[w] \leftarrow 1$.

(c) For all objects $u \in L_j$ do:

- For all objects $w \in Q(u, j)$, perform the operations of Steps 2(b)ii to 2(b)iv.:

(d) For all objects $w \in NewlyVisited$ do:

- Let $SimInt_{C,j}(v, w) \leftarrow RevIntCount[w]$.
- From $SimInt_{C,j}(v, w)$, compute (as in Section 4.3)

$$Temp \leftarrow \begin{cases} SimRatio_{C,j}(v, w), \\ SimCos_{C,j}(v, w), \text{ or} \\ SimCorr_{C,j}(v, w) \end{cases}$$

iii. If $Temp > SimValue[w]$ then update $SimValue[w] \leftarrow Temp$.

iv. Reset $VisitFlag[w] \leftarrow 0$, and delete $NewlyVisited \leftarrow NewlyVisited \setminus \{w\}$.

3. For all objects $w \in Visited$ do:

- Append the object-correlation ordered pair $Result \leftarrow Result \cup \{(w, SimValue[w])\}$.
- Reset $RevIntCount[w] \leftarrow 0$, and delete $Visited \leftarrow Visited \setminus \{w\}$.

4. Sort the ordered pairs of list *Result* in non-increasing order of the similarity values, and return the list. Ties can be broken arbitrarily, with the exception that v is given priority over any other object $w \neq v$ in S .

Figure 12: Pseudocode description of CES.